

Efficiently Reproducing Distributed Workflows in Notebook-based Systems

Talha Azaz¹, Raza Ahmad², Md Saiful Islam³, Douglas Thain³, Tanu Malik¹

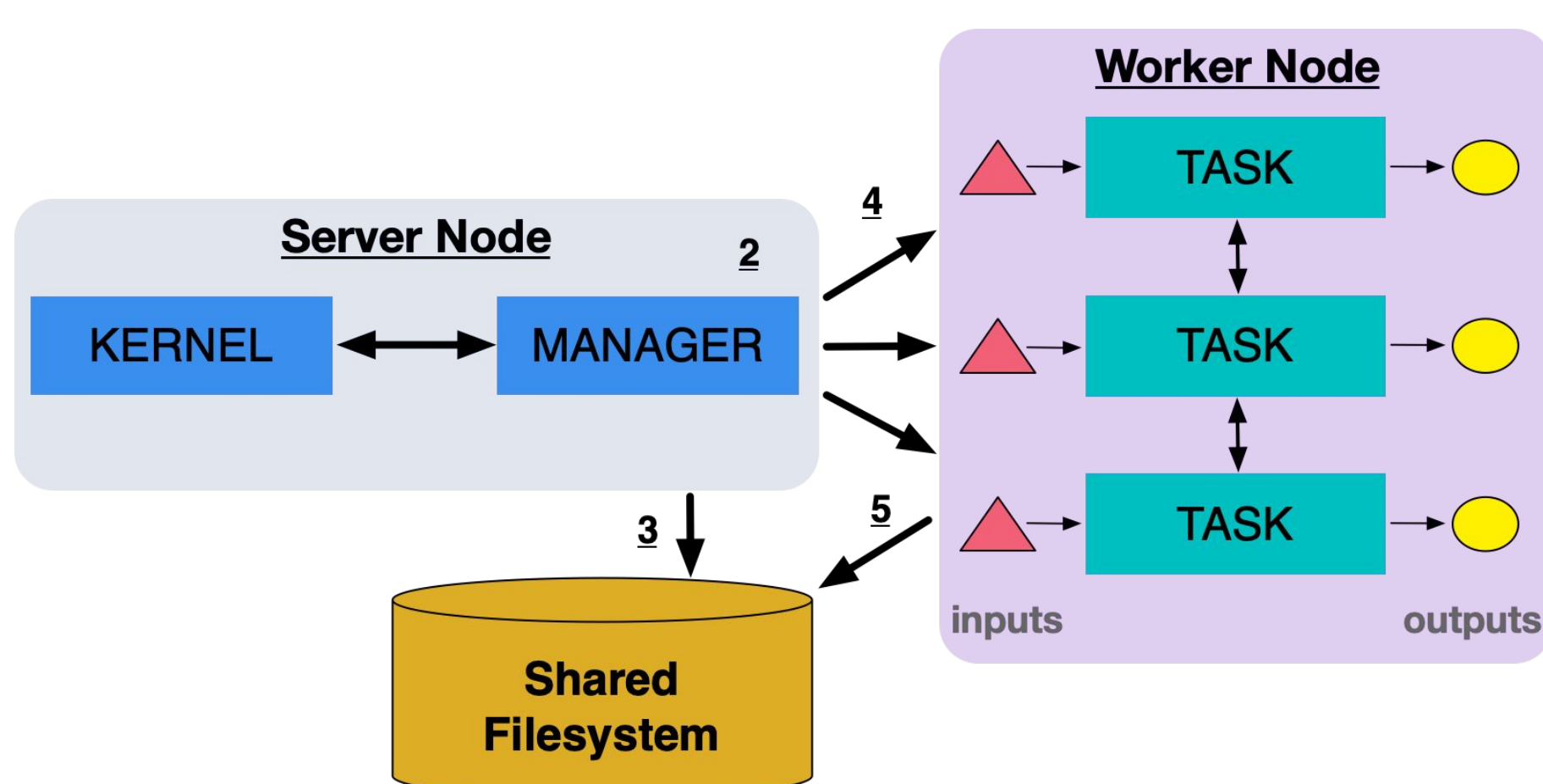
¹University of Missouri, ²Depaul University, ³University of Notre Dame



ABSTRACT

- Sharing and reproducing distributed workflows remains challenging, as even small code or parameter changes often force complete re-execution.
- We introduce **NBReplay**, an end-to-end system for efficient, reproducible execution of distributed workflows in notebooks. It consists of two kernels: audit and repeat, based on data-flow analysis across cells.
- The *audit kernel* avoids redundant computations through task level caching and incremental, cell-level checkpointing. The *repeat kernel* replays the cached tasks and reconstructs the checkpoints to enable partial re-execution of notebook cells that manage distributed workflow.
- NBReplay improves notebook re-execution by **2x-18x**, and enables portable, cross-site reproducibility of distributed workflows on HPC systems, while adding minimal overhead of execution time and storage space.

DISTRIBUTED EXECUTION IN NOTEBOOKS



PROBLEM DESCRIPTION

Consider the example notebook workflow to the right...

Scenario 1: When the *risk_score* formula in cell 2 is changed, cells 2 and 3 need to be re-executed. However, the distributed group by aggregation in cell 2 remains valid.

Scenario 2: When the notebook is shared to run on a new machine, if *risk_score* threshold in cell 3 is changed, the entire notebook needs to be re-executed. However, there is no change in cells 1 and 2 to be run again.

1. Load Data

```
from dask.distributed import Client
import dask.dataframe as ddf
client = Client("tcp://scheduler:8786")
df = ddf.read_parquet("run_24-06.parquet")
df = df.assign(anomaly = df["t"]-df["tb"])
raw_df = df
```

2. Worker Code

```
stats = (rdf.groupby(["region", "day"]).agg({
    "anomaly": ["mean", "std"], "p": "sum"}))
def risk_score(row):
    return 1.0*row["anomaly", "mean"]
daily_stats = stats.map_partitions(
    lambda p: p.assign(risk_score=p.apply(
        risk_score,axis=1)))
```

3. Distributed Workflow

```
high_risk = stats[stats["risk_score"] > 10.0]
result = high_risk.compute()
result.to_parquet("high_risk_24-06.parquet")
```

NBREPLAY METHODOLOGY

1. Cache distributed tasks including their code, inputs and outputs, and replay their outputs in the future for partial re-execution
2. Create incremental checkpoints to store intermediate workflow state after each cell execution which can be restored during re-execution.

NBReplay creates two Jupyter kernels:

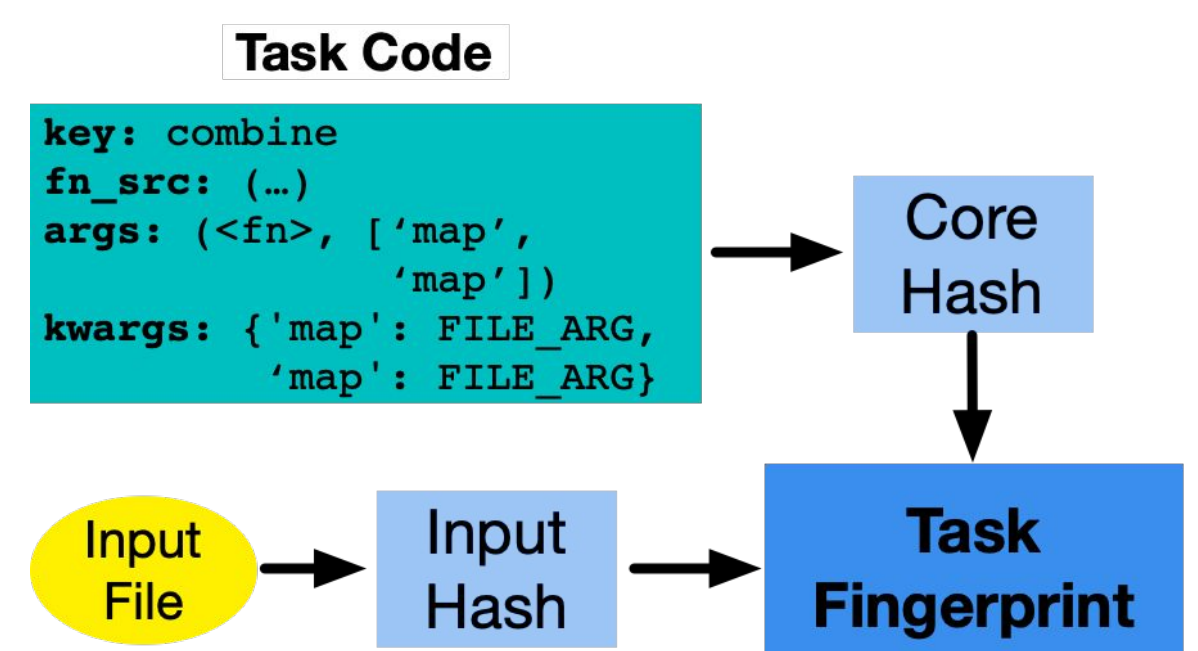
- **Audit** – for creating checkpoints and caching distributed tasks
- **Repeat** – for restoring checkpoints and replaying cached tasks

INCREMENTAL CHECKPOINTING

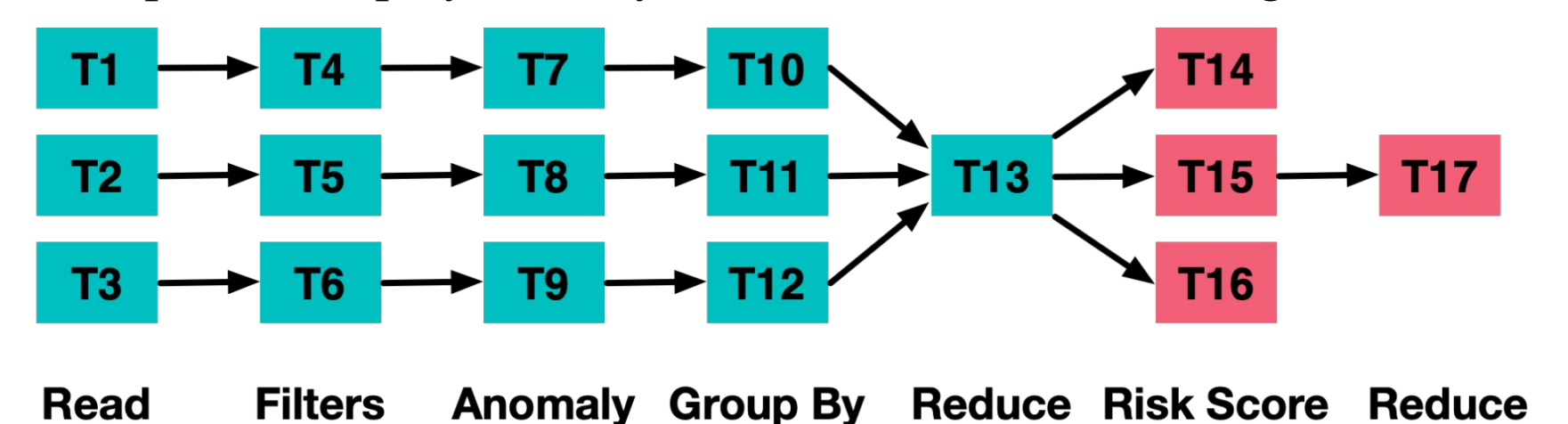
- Audit kernel performs application-level checkpointing of the Python session using static slicing and dynamic dependency tracking through data-flow analysis.
- Checkpoint includes variables, runtime objects, libraries, etc.
- Checkpoints are incremental – only the modified state changes are stored.
- Multiple data structures are maintained to track dependencies and references.

PARTIAL RE-EXECUTION

- A log and cache of distributed tasks is maintained, including their code, inputs, and outputs.
- New tasks are compared with existing tasks in the cache through a task **fingerprinting** mechanism. A cache hit results in replaying the cached output.

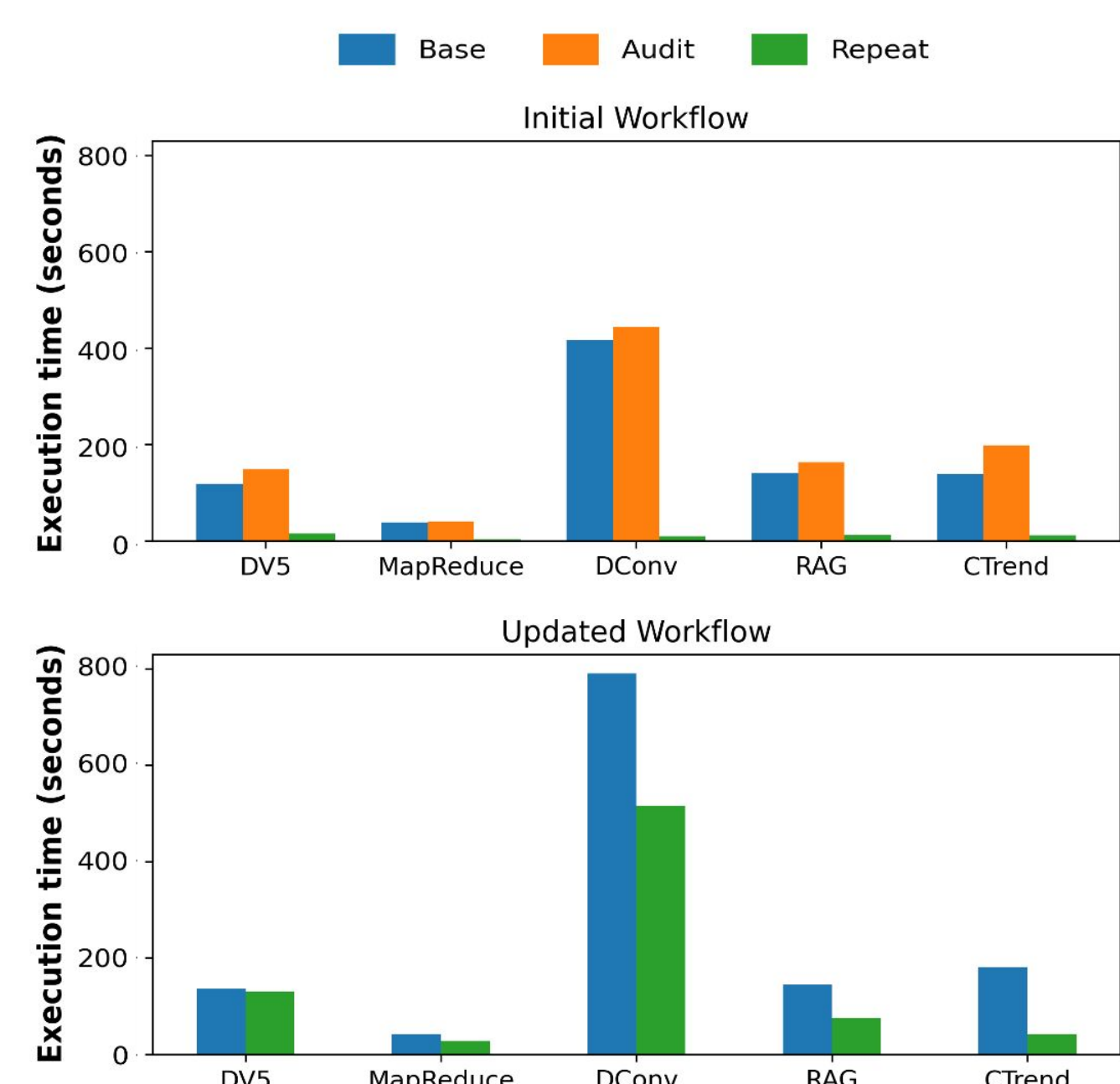


Following is the DAG of 17 tasks generated for the example notebook workflow using partial re-execution for Scenario 1. Tasks in green are found in the cache and their cached outputs are replayed. Only tasks in red are executed again.



EXPERIMENTS AND RESULTS

- We perform experiments on 5 real-world distributed notebook workflows on a SLURM cluster with 4 compute nodes using NFS4/XFS for cluster/local storage, with each node having 2.50GHz CPU and 4GB memory.
- We measure:
 1. **Overhead in checkpoint creation**
18% overhead in execution time during audit
 2. **Time savings during repeat execution without notebook change**
Avg performance gain of 18.4x, ranging from 8x to 46.2x
 3. **Time savings during repeat execution with notebook change**
Avg performance gain of 2.08x, ranging from 1.04x to 4.41x



NBREPLAY ARCHITECTURE AND WORKFLOW

